



Préparer la partie technique d'un entretien

Human**bo**oster
TRANSFORMEZ VOS CODES

Cyril COUFFE

Programme de la journée

Objectifs : travailler sur sa posture en test/entretien technique, comprendre comment ça marche, poser les bases pour s'entraîner, faire des simulations

Temps pédagogiques : échanges, présentation des différents dispositifs, entraînements aux tests techniques, mises en situation, simulation en one-to-one avec feedbacks individuels

La logique d'un entretien ou test technique

Temps d'échange :

- Pourquoi on fait des tests ou entretiens techniques pour recruter un développeur ou une développeuse ?
- C'est quoi le profil idéal d'un développeur/une développeuse ?

Les objectifs d'un entretien technique

- Réduire le risque d'une mauvaise embauche
- Comprendre comment vous réfléchissez
- Connaître l'état de vos compétences actuelles et leur adéquation avec le poste
- Comprendre les softs skills en lien avec différentes situations (blocage, manque de connaissance, travail en équipe, etc.)
- Savoir comment vous travaillez face à un problème (on fonce dedans, on réfléchit avant de se lancer, on effectue des recherches, on questionne le client sur le besoin en cas de doutes, etc.) ET votre capacité de raisonnement (logique, volonté d'optimisation, etc.)
- Connaître votre façon de travailler (respect de conventions d'écriture, attentif aux détails, volonté de s'adapter à l'entreprise et ses normes, etc.)

En complément d'un entretien technique

- Un entretien technique vient en réalité compléter d'autres façons de capter les informations dont l'entreprise pense avoir besoin pour décider :
 - Entretien RH : motivation, situation personnelle, soft skills
 - Portfolio personnel : Il est le plus souvent consulté avant l'entretien technique
 - Votre compte github/gitlab : En un clin d'œil, il est possible de se faire une idée de la productivité, la curiosité, et l'engagement d'une personne
 - Vos projets personnels : Ils sont essentiels pour un profil junior, car ils vous feront gagner beaucoup de crédibilité et montreront votre engagement
 - Recommandations par des pairs ou anciens collègues ou chefs
 - Etc.

Différentes façons de tester

- Des questions techniques en rafale sur des sujets précis (à l'oral)
- QCM (<https://welovedevs.com/fr/app/tests>)
- Correction/optimisation de code
- Exercice écrit (papier stylo ou sa version numérisée)
- Préparer un projet à rendre en quelques jours
- Faire 1 ou plusieurs exos en quelques heures (durée minutée) et rendre son travail via une adresse web ou un portail en ligne
- Faire du live coding devant 1 ou plusieurs personnes

Domaines techniques ciblés

- Conception de base de données
- Réalisation d'un algo pour résoudre un problème (typiquement des *parsers*, des exos qui testent votre réflexion et votre capacité à produire du code propre, maintenable et optimisé)
- API backend (type REST, SOAP, ou sans convention)
- Front (natif ou framework)
- Projet fullstack web
- Virtualisation par conteneurisation et déploiement
- Tests unitaires et documentation technique
- Etc.

Quand ça se passe mal

- Vous allez bloquer sur certaines questions
- Vous allez paniquez
- Vous allez entrer en mode de blocage mental
- Etc.

Que faire ?

Les pièges à éviter quand ça coince

- Dénigrer son ancien poste, sa formation, son stage quand vous ignorez la réponse
- Inventer des réponses (il est pire de dire n'importe quoi sur un sujet technique que de dire qu'on ignore)
- Paniquer
- Etc.

Plutôt aller vers d'autres stratégies

- Citer les bases de connaissances qu'on a sur le sujet pour montrer qu'on a tout de même un terrain d'apprentissage
- Dire que vous ne savez pas
- N'hésitez pas à poser des questions pour montrer que vous êtes en posture d'acquisition des connaissances à tout moment. Même si l'entretien n'est pas concluant, vous aurez appris quelque chose !
- Un test ou entretien technique « raté » n'en est pas forcément un. Vous pouvez avoir la sensation d'échouer l'exercice tout en donnant des gages de réassurance suffisants ! L'important le plus souvent n'est pas le résultat du test mais le chemin de réflexion !

Préparation : en amont de l'entretien

- Préparez-vous pour des questions techniques précises
- Entraînez-vous sur des plateformes à résoudre des problèmes de code
- Maintenez votre portfolio à jour
- Gérez votre Github avec des projets persos

Préparation : en cours d'entretien

- Restez humble
- Montrez votre intérêt pour le poste, la culture de travail
- Restez calme
- Ne paniquez pas
- Ne vous jugez pas vous-même
- En fin d'entretien, n'hésitez pas à demander un feedback

Entrainement 1

- Choisissez une offre d'emploi / de stage
- Partez du principe que vous avez réussi la partie RH de l'entretien, et que vous allez être reçu en entretien technique qui se déroulera en 2 phases :
 - Questions libres sur divers sujets
 - Une partie live coding
- Organisez votre préparation en amont de cet entretien :
 - Ciblez les bonnes ressources d'entraînements
 - Imaginer un mini-projet que vous feriez en amont pour vous entraîner
 - Listez des questions qui pourraient vous être posées

Effectuez le travail avec sérieux, nous ré-utiliserons ce matériel !

Entrainement 2

- Mettez-vous par groupe de 2. Vous allez alterner le rôle de la personne qui passe l'entretien et de celle qui le fait passer.
- Chaque personne doit transmettre le travail réalisé pendant le 1^{er} entraînement, notamment l'offre d'emploi et les questions préparées.
- Temps de préparation : en lisant l'offre d'emploi, la personne jouant le rôle du recruteur devra compléter/préparer des questions techniques à poser
- Temps d'échange : à tour de rôle, vous jouerez le rôle du recruteur puis de la personne qui passe l'entretien. Le but est de vous faire vous entraîner à répondre à des questions techniques. Pour certaines, vous ignorerez les réponses, et c'est normal.

Entrainement 3

- Mettez-vous par groupe de 2 ou 3. Vous allez alterner le rôle de la personne qui passe l'entretien et d'observateur.
- Un observateur doit choisir une épreuve à réaliser (voir diapo suivante). La personne passant l'entretien devra le réaliser en live coding sans aide extérieure (évidente) d'un observateur.
- Comme dans une live coding classique, la personne devra expliquer à voix haute ce qu'elle fait.
- Alternez les rôles afin d'avoir été dans les 2 situations au moins 1 fois.

Entrainement 3

2 exos que je propose ici :

<https://cyril-couffe.notion.site/Exos-algos-1bc8815dedf28035bbfdd4170c3a520c?pvs=4>

Conseils pour gérer le live coding

- Commentez votre code
- Gare à l'indentation et la lisibilité du code
- Poser votre réflexion en amont avant de vous lancer (via des commentaires)
- Découpez votre logique en fonction unitaire appelée par une fonction principale
- Faites la vérification de réussite ou d'échec via un test unitaire
- Gare au nommage des variables et fonctions (soyez explicite, full english)
- Pensez à la gestion des erreurs et leur propagation (try catch ou autre logique)
- Pensez à l'optimisation du code une fois celui-ci fonctionnel (éviter des traitements inutiles, early returns, boucles inutiles, etc.)